



Innovatech Solutions

Caso semestral DSY1106

John Beiza
Dante Torres
Profesora Viviana Poblete
Sección 001D

Índice

[Contexto del caso](#)

[El cliente](#)

[El proyecto](#)

[Requerimientos detectados](#)

[Patrón de arquitectura](#)

[Esquema](#)

[Justificación](#)

[Interacción entre microservicios](#)

[Esquema](#)

[Justificación](#)

[Contenedores](#)

[Diagrama](#)

[Justificación](#)

[Evaluación general previa](#)

[Conclusión](#)

[Referencias](#)

Contexto del caso

El cliente

Innovatech Solutions es una empresa dedicada al desarrollo de software a medida y consultoría para organizaciones de diversos sectores, e incluyendo organizaciones tanto públicas como privadas.

La empresa cliente busca ofrecer soluciones tecnológicas de alta calidad, por lo cual se emplean profesionales de especialidades varias y se forman equipos multidisciplinarios, los cuales incluyen desarrollo de software, gestores de proyecto y consultores, entre otros.

Los equipos de Innovatech se encuentran dispersos geográficamente, y presencian dificultades a la hora de gestionar sus recursos, acceder a información fidedigna y actualizada, y medir y manejar adecuadamente el progreso dentro de los objetivos de trabajo.

Gracias al crecimiento acelerado de la empresa presenciado en los últimos años, el flujo de trabajo se ha visto en necesidad de una mejor organización y comunicación entre equipos, para lo cual se formula el presente proyecto.

El proyecto

Se trata del desarrollo de una plataforma construida bajo una arquitectura moderna basada en microservicios, con tal de asegurar escalabilidad y flexibilidad a largo plazo, y mejorar el manejo de recursos de la organización.

La solución considerará tres aspectos principalmente:

1. Gestión de proyectos.
2. Gestión de recursos y colaboración.
3. Monitoreo y analítica.

En base a estos aspectos generales, se realizó un levantamiento de requerimientos generales, con tal de mejor visualizar las necesidades del cliente y realizar así una planeación de arquitectura alineada debidamente con su visión.

Requerimientos detectados

Funcionales

1. Gestión de Proyectos

a. Proyectos

- i. Planificación: Debe permitir la subida de documentos, imágenes, y texto, y la edición de este último.
 - 1. El software permitirá la carga de documentos de sólo ciertos formatos (por definir).
- ii. Ejecución: Debe permitir modificaciones a cada aspecto de un proyecto.
- iii. Seguimiento: Se debe poder definir el estado de un proyecto, visualizando fechas utilizando una vista de calendario.

b. Tareas

- i. Definición: Debe permitir la definición de tareas dentro de un proyecto, utilizando un formato de checklist.
- ii. Asignación: Debe permitir la asignación de responsables para cada tarea.
- iii. Control de estados de avance: Debe permitir la definición y visualización del estado de una tarea.

2. Gestión de Recursos

- a. Asignación de profesionales: Debe facilitar la asignación de profesionales a los distintos proyectos y tareas, visualizando su nombre, apellido, contacto y proyecto o asignación actual.
- b. Disponibilidad del recurso humano: Debe mostrar la disponibilidad de cada profesional mediante un listado que defina su estado y otros datos pertinentes.
- c. Colaboración
 - i. Subida de avances: Todo usuario debe ser capaz de subir avances o actualizar tareas según le corresponda.

3. Monitoreo y Analítica

a. Panel interactivo

- i. Monitorear desempeño: El panel debe mostrar indicadores clave de desempeño (KPI).
- ii. Métricas de avance: El panel debe tener un apartado para métricas de avance, organizadas según corresponda.
- iii. Utilización de recursos: El panel debe mostrar la utilización de recursos clave mediante cifras o gráficos.

- iv. Estado general: El panel debe disponer de una sección que muestre un resumen general del desempeño de la empresa.

No Funcionales

1. Interfaz

- a. Debe contar con una interfaz de usuario intuitiva y responsiva para toda área del software.
- b. Fuente y colores: La fuente debe ser fácilmente legible, y los colores amigables para la vista, cuidando criterios de visibilidad HTML.
- c. Apariencia de la información: Tanto el texto como las imágenes y enlaces deben estar claramente señalados.
- d. Layout: Todo el sistema debe estar distribuido de manera armoniosa, tal que el usuario pueda llegar fácilmente a lo que necesita. Los componentes deben estar señalados de manera clara y legible.
- e. Responsividad: El sistema debe ser responsivo según el tamaño del dispositivo de acceso, ajustando la distribución y tamaño de los elementos según sea necesario.

2. Actualizaciones de proyecto

- a. Al subir o modificar un archivo o tarea, el cambio debe verse reflejado de inmediato en la página correspondiente, demorando un máximo de 3 segundos en visualizarse.
- b. El estado de cada proyecto debe verse claramente señalado de manera visual, agilizando el flujo de trabajo.

3. Tiempos de espera

- a. El usuario debe ser capaz de ver el estado de carga de cualquier componente del sistema, sea subida de archivos, cambio de registros, o carga de una página.
- b. Luego de la carga inicial de los datos de una página, estos deben demorarse un máximo de 5 segundos en recargarse, exceptuando casos extremos donde se modifique un gran volumen de registros o se actualice de alguna manera el software.

4. Construcción HTML y en componentes

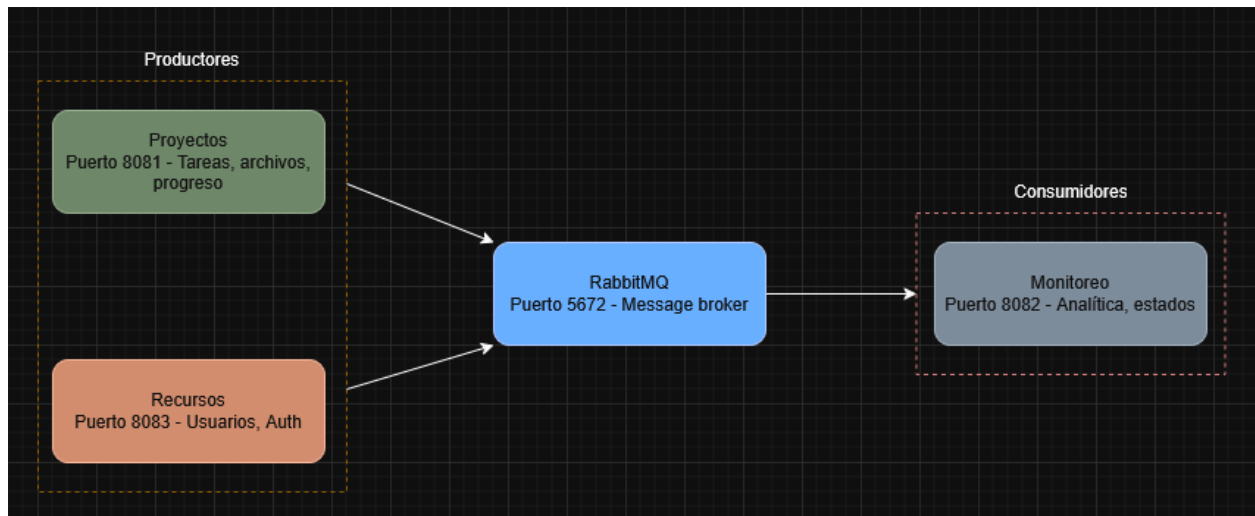
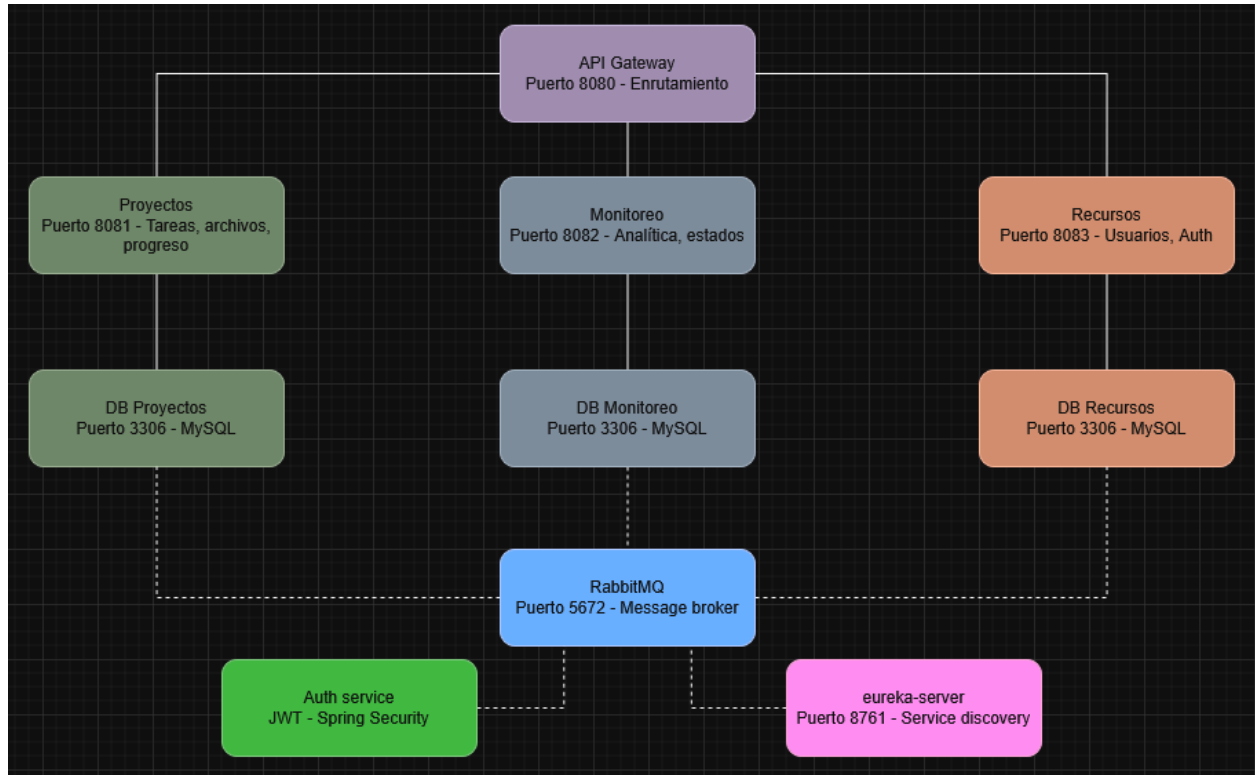
- a. La construcción del sitio web debe seguir normativas y buenas prácticas de diseño web, como lo es el etiquetado semántico de cada sección relevante.
- b. Se diseñará el sitio utilizando componentes repetitivos (footer, header, navbar, etc), con tal de minimizar el código y los tiempos de carga.

5. Arquitectura

- a. Microservicios: El sistema y sus requerimientos se construirán bajo una arquitectura que incorpora microservicios, cuidando la escalabilidad y flexibilidad de mantención del servicio.
 - i. Desacoplamiento: Los servicios creados serán independientes entre sí, permitiendo mejoras individuales sin comprometer el resto del sistema.
 - b. Basada en eventos: El sistema incorporará una arquitectura basada en eventos para las funcionalidades en tiempo real.
6. Comunicación en el sistema
- a. API Gateway: Se diseñará un API Gateway para gestionar las peticiones entre el frontend y los microservicios.
7. Patrones de diseño
- a. Repository Pattern: Se implementará para abstraer la lógica de acceso a las bases de datos (utilizando JPA). Esto centralizará las consultas y asegurará que los microservicios (como el de proyectos o recursos humanos) no dependan directamente de la base de datos, facilitando su mantenimiento y persistencia.
 - b. Factory Method: Se utilizará para la creación estandarizada de instancias de objetos complejos dentro del backend (por ejemplo, diferentes tipos de proyectos o roles de usuario). Esto permitirá que el sistema sea escalable si Innovatech decide agregar nuevas categorías en el futuro sin modificar el código base.
 - c. Circuit Breaker: Se aplicará en la comunicación entre el API Gateway/BFF y los microservicios internos. Su función será detectar fallos en servicios específicos y cortar la conexión temporalmente, evitando que un error aislado provoque una caída en cascada de toda la plataforma y garantizando la continuidad operativa.
8. Rendimiento
- a. Múltiples solicitudes: El backend debe ser capaz de soportar múltiples solicitudes concurrentes sin que el rendimiento se vea comprometido.

Patrón de arquitectura

Esquema



Justificación

Al construir la solución, se decidió trabajar combinando dos patrones de arquitectura distintos: de microservicios y basada en eventos.

La arquitectura basada en microservicios responderá a la necesidad de un sistema escalable y flexible. Se separaron los distintos microservicios según su función dentro del sistema, junto con otros componentes esenciales de un software de calidad que cumpla con los estándares propuestos.

API Gateway

Debido a la arquitectura de microservicios, se ve necesario implementar una API Gateway que cumpla con la función de ser la puerta entre el cliente y los servicios, ofuscando estos últimos y aumentando la seguridad del sistema, además de seguir buenas prácticas.

Mediante la implementación de una API Gateway, será posible acceder a todos los microservicios desde un mismo puerto, haciendo de este un paso fundamental al trabajar con microservicios. Facilita también la personalización del contenido, pudiendo mostrar servicios específicos a clientes específicos.

Es un punto crucial de cualquier sistema basado en microservicios, promoviendo seguridad y encargándose del enrutamiento.

MS: Proyectos

Se trata de un servicio que incluirá la gestión de proyectos, es decir, su creación y manipulación dentro de la base de datos, además de la creación de tareas y archivos relacionados a dichos proyectos. Para estos propósitos, se proponen los siguientes endpoints:

- `/api/proyectos; /api/proyectos/{id}`: CRUD de proyectos.
- `/api/proyectos/tareas; /api/proyectos/{id}/tareas`: CRUD de tareas asignadas por proyecto.
- `/api/proyectos/archivos; /api/proyectos/{id}/archivos`: CRUD de archivos relacionados al proyecto.

MS: Monitoreo

Este servicio se encargará de la analítica y el control de estado de proyectos y recursos. A través del servicio de monitoreo será posible la visualización de reportes y resúmenes, un requerimiento de alta importancia para el cliente. Además, será el foco central de la arquitectura basada en eventos, comunicándose con RabbitMQ para generar notificaciones y alertas en tiempo real.

- /api/monitoreo/reportes; /api/monitoreo/reportes/{id}: CRUD reportes.

MS: Recursos

El servicio de recursos tiene como misión principal el facilitar la gestión del recurso de personas, por lo cual se encarga del manejo y gestión de usuarios, incluyendo la autorización y verificación de credenciales.

- /api/recursos/usuarios; /api/recursos/usuarios/{id}: CRUD usuarios.
- /api/auth: endpoint dedicado a la validación de credenciales (login).

Base de datos: MySQL

MySQL es una base de datos relacional open source, que fue escogida como la más apropiada para el proyecto por las siguientes razones:

- Costo: Al ser open source, MySQL es completamente gratis para utilizar en cualquier proyecto.
- Reputación: MySQL tiene una buena reputación en el mundo del desarrollo de software, contando con ya 30 años de longevidad.
- Soporte: Gracias a la antigüedad de MySQL, existe extensa documentación y gran cantidad de recursos en línea que son útiles a la hora de lidiar con errores.
- Facilidad: MySQL es simple de usar y ofrece amplia compatibilidad con diversas plataformas y lenguajes de programación, incluyendo Java, Python, PHP, y JavaScript.

Message Broker: RabbitMQ

RabbitMQ es un message broker de código abierto, cuyo propósito principal dentro del sistema será facilitar las funciones de notificaciones y cambios de estado en tiempo real. Se encargará de toda petición y función asincrónica realizada dentro del sistema.

Seguridad: JWT Spring Security

Para el ángulo de seguridad, se trabajará el acceso a ciertas partes sensibles del sistema utilizando autorización basada en JWT (JSON Web Tokens) con la ayuda del framework Spring Security.

Spring Security se acopla perfectamente al proyecto, siendo este ya construido en Spring. Además, gran parte del manejo de proyectos dentro de la compañía, y la existencia de distintos roles en ella, demandan diferenciación entre usuarios. El uso de JWT permitirá delimitar los rincones del sistema a los cuales estos podrán acceder de forma satisfactoria.

Infraestructura: Eureka Server

Eureka server es una herramienta que actúa como registro de servicios, teniendo información sobre cada uno. Su principal propósito en el sistema es facilitar la interacción entre microservicios, ya que permite su registro de manera automática, sin necesidad de declarar explícitamente direcciones o rutas.

El objetivo al usar Eureka para el registro de microservicios es ahorrar trabajo y código al conectar servicios entre sí, ya que centraliza esta información y la actualiza de manera dinámica.

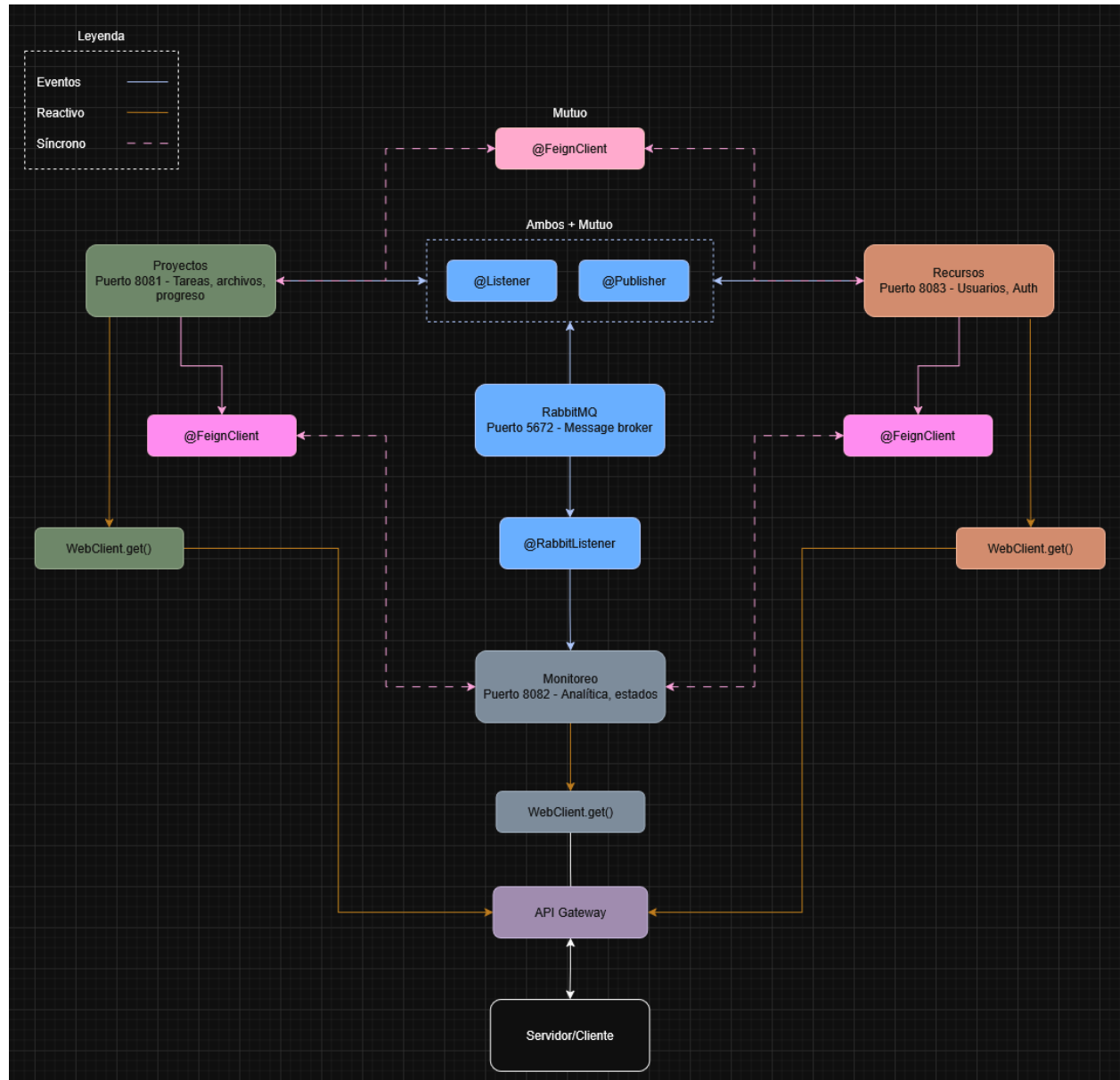
Patrón de arquitectura: EDA

Finalmente, para complementar la arquitectura basada en microservicios, se trabajará bajo una EDA: Event Driven Architecture. Esta arquitectura de software tiene su enfoque en los eventos del sistema, los cuales son definidos como cualquier acción o cambio significativo que ocurra durante su funcionamiento.

Mediante el uso de RabbitMQ como message broker, será posible reflejar cambios en tiempo real y enviar notificaciones relevantes durante la gestión de proyectos. La función principal relacionada a esta decisión recae en la obtención de actualizaciones automáticas del lado de los jefes de proyecto, quienes podrán ver el progreso de cada tarea y proyecto en su dashboard a medida que los miembros de su equipo realizan cambios.

Interacción entre microservicios

Esquema



Justificación

Para asegurar el rendimiento, la escalabilidad, y cumplir con el requerimiento de monitoreo en tiempo real, la plataforma ideada para Innovatech implementará un enfoque de comunicación híbrido entre sus microservicios, combinando llamadas síncronas, asíncronas y una arquitectura orientada a eventos.

Se utilizarán las siguientes herramientas del ecosistema Spring Boot:

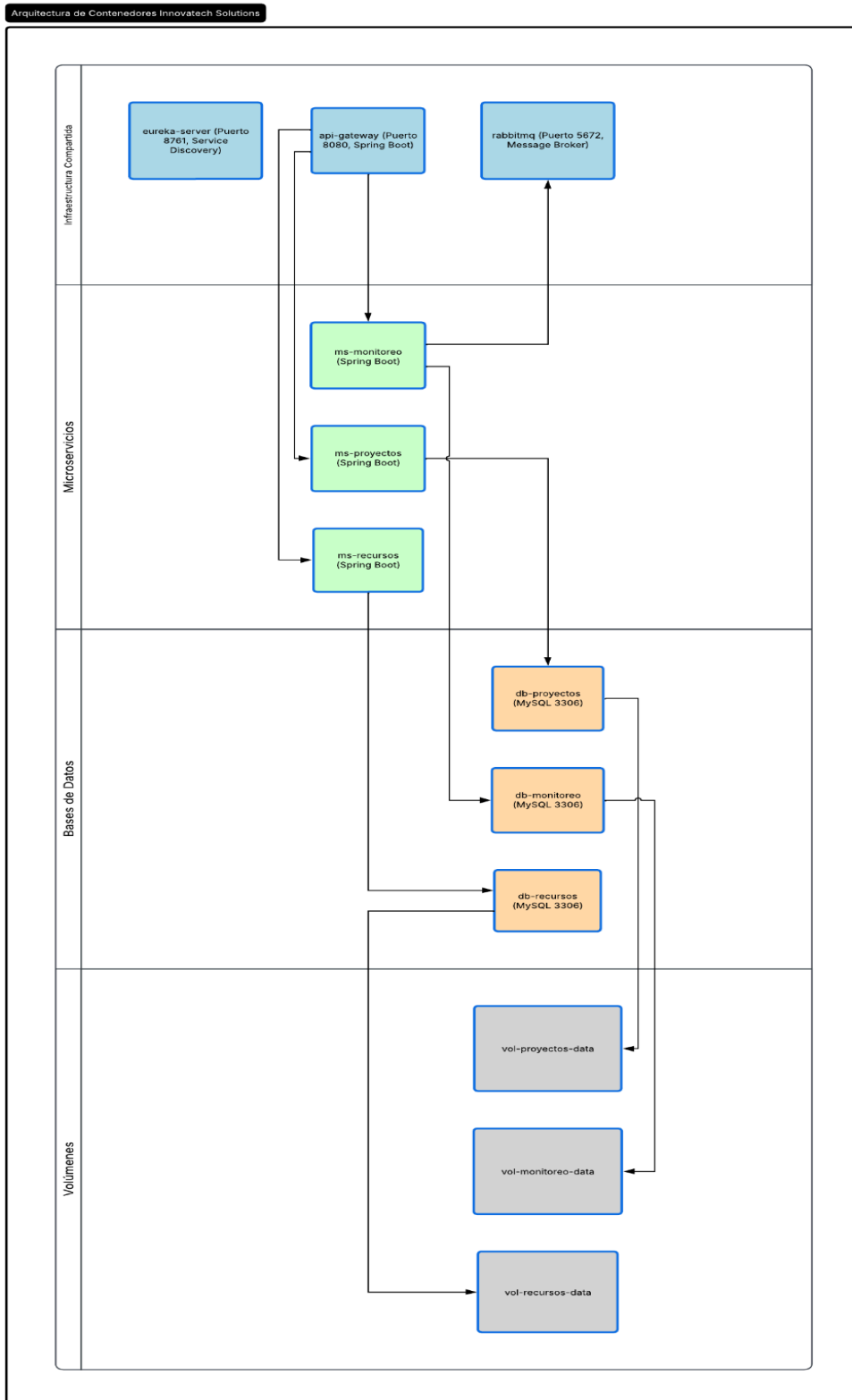
- **Comunicación Síncrona (OpenFeign):** Se implementará OpenFeign para las interacciones directas y de resolución inmediata entre microservicios. Por ejemplo, cuando el servicio de Gestión de Proyectos necesite validar la disponibilidad de un empleado antes de asignarle una tarea, realizará una petición síncrona al servicio de Gestión de Recursos. Esto garantiza que los datos sean consistentes antes de ejecutar una acción crítica.
- **Comunicación Asíncrona y Reactiva (Spring WebClient):** Para evitar cuellos de botella en operaciones que requieren consultar grandes volúmenes de datos, se utilizará WebClient. Al ser una herramienta no bloqueante (non-blocking) basada en Spring WebFlux, permite que los microservicios realicen peticiones sin quedarse en espera de la respuesta. Esto es fundamental para el API Gateway, permitiéndole manejar múltiples solicitudes concurrentes de los usuarios sin comprometer el rendimiento general del sistema.
- **Arquitectura Orientada a Eventos (RabbitMQ):** Para cumplir con la necesidad de monitorear el progreso y los indicadores clave (KPIs) en tiempo real, se integrará RabbitMQ como Message Broker. Cuando ocurra un cambio de estado en una tarea o proyecto, el servicio correspondiente publicará un evento en RabbitMQ. El servicio de Monitoreo y Analítica estará suscrito (mediante `@RabbitListener`) a estos eventos, procesándolos de inmediato para actualizar los paneles y métricas de avance.

Aporte a la eficiencia técnica y operativa:

Este modelo híbrido garantiza un alto nivel de desacoplamiento. Si el servicio de Monitoreo se encuentra temporalmente inactivo o bajo alta carga, el servicio de Proyectos puede seguir funcionando y publicando eventos en RabbitMQ sin fallar, los cuales serán procesados una vez que el servicio de monitoreo se restablezca. Además, el uso de WebClient optimiza el consumo de hilos en el servidor, mejorando drásticamente la eficiencia operativa frente a la alta concurrencia.

Contenedores

Diagrama



Justificación

Para el desarrollo e implementación de la plataforma de Innovatech Solutions, se ha optado por una arquitectura basada en contenedores utilizando **Docker**, orquestada inicialmente con Docker Compose y diseñada para escalar mediante **Kubernetes** en producción.

El ecosistema se despliega sobre una red interna (innovatech-net) y se compone de microservicios en Spring Boot, un API Gateway como único punto de acceso (puerto 8080), y bases de datos MySQL independientes.

Esta arquitectura se justifica en base a los siguientes factores:

- **Seguridad y Privacidad:** Al utilizar una red aislada de Docker, las bases de datos y los microservicios internos no están expuestos a internet. Todo el tráfico externo pasa obligatoriamente por el API Gateway, previniendo accesos no autorizados a los datos de la empresa y sus clientes.
- **Sostenibilidad:** Los contenedores comparten el kernel del sistema operativo anfitrión, a diferencia de las máquinas virtuales tradicionales. Esto optimiza drásticamente el uso de hardware (CPU y RAM), reduciendo el consumo energético de los servidores y minimizando la huella de carbono del proyecto.
- **Escalabilidad (Kubernetes):** La contenedorización prepara el sistema para el auto-escalado. Si el módulo de "Monitoreo" experimenta alta demanda procesando métricas, Kubernetes puede levantar réplicas adicionales solo de ese contenedor, sin malgastar recursos en replicar toda la plataforma.
- **Persistencia de Datos:** Se implementaron volúmenes de Docker acoplados a cada instancia de MySQL. Esto garantiza la integridad y persistencia de la información crítica frente a cualquier reinicio o caída de los contenedores.

Evaluación general previa

Diseño

El diseño escogido fue pensado considerando los requerimientos del cliente, cuidando principalmente los aspectos técnicos sobre la escalabilidad, flexibilidad, y capacidad de desacoplamiento. Además, se consideraron los recursos del equipo, principalmente el tiempo y la complejidad de implementación.

El cliente especificó el uso de microservicios, por lo que el sistema fue planeado en torno a esto, determinando cómo desacoplar las funciones requeridas y cómo proyectar la interacción entre los microservicios definidos.

Asimismo, considerando todo lo anterior, se definen las herramientas a utilizar durante el desarrollo.

Herramientas

Se enlistan las tecnologías a utilizar, con una breve descripción y justificación.

- **Spring Boot:** Se utilizará el framework de Spring, así como múltiples componentes y dependencias relacionadas a este, debido a su facilidad de implementación y amplia oferta de funcionalidades. Además, el ecosistema Spring ofrece prácticamente todo lo necesario para el proyecto, incluyendo, pero no limitándose a: Spring Security, Spring WebClient, Spring Boot Eureka Server, entre otros.
- **RabbitMQ:** Se utilizará esta herramienta como message broker, debido a su reputación como un componente suficientemente robusto, pero igualmente relativamente sencillo de implementar.
- **Docker:** Docker es el estándar para el trabajo con contenedores. Al trabajar con microservicios, es crucial que cada uno tenga su propio entorno debidamente configurado y fácilmente replicable, lo cual será logrado mediante el uso de contenedores.
- **Kubernetes:** Al tener múltiples microservicios, y, por consiguiente, múltiples contenedores, es crucial el poder gestionarlos de manera eficiente. Kubernetes permitirá asegurar la gestión, automatización y escalabilidad óptima de los contenedores.
- **MySQL:** MySQL es el motor de base de datos de elección, debido a su facilidad de uso, extensa documentación, e historial altamente confiable.

Requerimientos destacables

En cuanto a las funciones, se dio especial hincapié al monitoreo de resultados en tiempo real, dado que este aspecto es de gran importancia para el cliente. Para este fin, se propone la arquitectura basada en eventos, combinada con los microservicios, lo cual lleva a la implementación de RabbitMQ.

Al ser una empresa que maneja proyectos y debe asegurar consistencia en su progreso para la obtención de resultados de calidad, se da especial cuidado a la consistencia y seguridad de los datos, con tal de facilitar la colaboración entre los equipos. Por esto, se utiliza OpenFeign para la comunicación entre microservicios, en conjunto con Spring Security y MySQL para cumplir con requisitos de seguridad e integridad de datos.

Finalmente, el trabajo con microservicios debidamente realizados requiere del uso de una API Gateway, la cual permite la realización de peticiones a las API sin comprometer su seguridad mediante el enrutamiento y redirección de puertos.

Conclusión

Se concluye que el trabajo realizado involucra la investigación y entendimiento sobre distintas tecnologías esenciales a la hora de desarrollar software de calidad, el cual debe considerar varios aspectos, tales como: escalabilidad, seguridad, optimización, y coherencia, entre otros.

Al diseñar el sistema y sus distintos componentes previo del trabajo de codificación en sí, se logra un entendimiento más profundo de este, logrando asegurar el cumplimiento de requisitos, algo esencial a la hora de entregar productos que satisfacen al cliente y contribuyen a la experiencia y reputación del equipo desarrollador.

Finalmente, se considera que el proceso de elección de arquitectura(s) e implementación de recursos visuales mediante diagramas, así como el ejercicio de justificación de decisiones, resulta favorable a la hora de entregar un producto de calidad de manera consciente, y es un componente esencial dentro del desarrollo de software.

Referencias

Docker Inc. (2026). Docker Documentation. Recuperado de <https://docs.docker.com/>

Kubernetes Authors. (2026). Kubernetes Documentation. Recuperado de <https://kubernetes.io/docs/>

RabbitMQ. (2026). RabbitMQ Documentation. Broadcom. Recuperado de <https://www.rabbitmq.com/documentation.html>

Spring. (2026). Spring Boot Reference Documentation. VMware. Recuperado de <https://docs.spring.io/spring-boot/docs/current/reference/html/>

Spring. (2026). Spring WebFlux (WebClient). VMware. Recuperado de <https://docs.spring.io/spring-framework/reference/web/webflux-webclient.html>

Erickson, J. (2024, 29 agosto). MySQL: Understanding what it is and how it's used. Recuperado de <https://www.oracle.com/uk/mysql/what-is-mysql/>

Pattern: API Gateway / Backends for Frontends. (s. f.). Recuperado de <https://microservices.io/patterns/apigateway.html>

Spring Security. (s. f.). Recuperado de <https://spring.io/projects/spring-security>

Salgado, L. G. (2025, 11 febrero). Eureka Server: su papel en la arquitectura de microservicios. Recuperado de

<https://keepcoding.io/blog/que-es-eureka-server-en-la-arg-de-microservicio/>

GeeksforGeeks. (2025, 31 octubre). Spring Boot Eureka Server. Recuperado de <https://www.geeksforgeeks.org/advance-java/spring-boot-eureka-server/>